A TIME COMPLEXITY OF APPROXIMATION

Theorem A.1. *In the worst case, the time complexity of ApproxSFRS is* $O(KM \log M(1 + \log |\mathcal{S}_\forall| - \log M))$.

Proof. ApproxSFRS uses the same branch and bound search scheme as ExactSFRS. However, because $Q.size$ is limited to M, we can separate the search process into two phases:

- *Phase 1:* $Q.size < M$. In this phase, the search process is the same as ExactSFRS. Each branch and bound steps increase the $Q.size$ by 1 until $Q.size = M$.
- *Phase 2:* $Q.size = M$. When Q is full, splitting the best search space will cause $Q.size = M + 1$. Thus, every time a new search subspace is insert into Q, we also need to discard the worst search space in Q to keep $Q.size = M$. This phase ends when N results are found.

For Phase 1, the branch and bound steps repeat for M times, each of which costs $O(K)$ time. Moreover, the time complexity of maintaining Q (a min/max heap) is $O(\log M)$. Therefore, the worst-case time complexity of Phase 1 is $O(KM \log M)$. We next proof the worst-case time complexity of Phase 2.

We represent the search process as a traversal process on a full binary tree, where a tree node stands for a search space and the child nodes of a node denote its search subspaces. The root is the entire search space $\mathcal{S}_\forall$ and the leaf nodes are candidate regions. Each node on the search tree is associated with a distance lower bound. We denote the depth of a node $\mathcal{S}$ on the tree as $\mathcal{S}.depth$. The total depth of the tree is $\log |\mathcal{S}_\forall|$.

During the search process, a result region is found when a leaf node is retrieved from Q, i.e. $\mathcal{S}'.depth = \log |\mathcal{S}_\forall|$. Therefore, the best (i.e., fastest) case of finding a result is that

- In the branching step, the optimal space $\mathcal{S}'$ has the largest depth, i.e., $\mathcal{S}'.depth \geq \mathcal{S}.depth, \forall \mathcal{S} \in Q$.
- In the abandoning step, the worst space (denoted as $\mathcal{S}''$) has the smallest depth, i.e., $\mathcal{S}''.depth \leq \mathcal{S}.depth, \forall \mathcal{S} \in Q$.

This can guarantee that Phase 2 ends within $\log |\mathcal{S}_\forall|$ steps (i.e., the height of the tree), which is the best case of the search process.

Therefore, the worst case can be formulated as

- In the branching step, the optimal space $\mathcal{S}'$ has the smallest depth, i.e., $\mathcal{S}'.depth \leq \mathcal{S}.depth, \forall \mathcal{S} \in Q$.
- In the abandoning step, the worst space (denoted as $\mathcal{S}''$) has the largest depth, i.e., $\mathcal{S}''.depth \geq \mathcal{S}.depth, \forall \mathcal{S} \in Q$.

As we keep branching $\mathcal{S}$ with small depth and abandoning $\mathcal{S}$ with large depth, all the $\mathcal{S} \in Q$ have the same depth at a certain point of Phase 2. Base on this, Phase 2 can be divided into two sub-phases:

- *Phase 2.1:* The search process keeps branching
$$\mathcal{S}' = \arg\min_{\mathcal{S} \in Q} \mathcal{S}.depth,$$
and keeps abandoning
$$\mathcal{S}'' = \arg\max_{\mathcal{S} \in Q} \mathcal{S}.depth,$$
until $\forall \mathcal{S} \in Q$ have the same depth.
- *Phase 2.2:* After all the search spaces have the same depth (denoted as $d_\mathcal{S}$), We branch each of them one by one and generate another set of spaces that also have the same depth $d_\mathcal{S} + 1$.

In the worst case, Phase 2.1 takes another M steps to end, whose complexity is $O(KM \log M)$. Phase 2.2 can be considered as a binary search on each of the M nodes, whose depth is at least $\log M$. Thus, the time complexity of Phase 2.2 is $O(KM \log M(\log |\mathcal{S}_\forall| - \log M))$. The total complexity of Phase 2 is $O(KM \log M(1 + \log |\mathcal{S}_\forall| - \log M))$.

The total complexity of ApproxSFRS is $O(KM \log M(1 + \log |\mathcal{S}_\forall| - \log M))$. □